

Commands

Commands und ICommand

- Abstrakte, entkoppelte Form eines Event als Property im DataContext (ermöglicht Bindungen)
- Automatisches Aktivieren/Deaktivieren (CanExecute)
 - z.B. Copy nur wenn Text selektiert
- Diversere vordefinierte Standard-Commands (System.Windows.Input.ApplicationCommands)
 - Vordefinierte ausführbare Commands: Cut, Copy, Paste, ...
 - Vordefinierte Commands ohne eigene Logik: Close, Delete, Find, Open, ...
- Individuelle Custom-Commands (ICommand)
 - Methoden:
 - CanExecute
 - Execute
 - Event
 - CanExecuteChanged

Commands bieten eine Möglichkeit, Benutzeraktionen mit Logik in der Anwendungslogik zu verbinden, ohne dass der Code direkt in der UI-Elementklasse enthalten sein muss. So kann die Logik in den entsprechenden Daten- oder ViewModel-Klassen enthalten sein.

Das ICommand-Interface ist das Kernstück des Command-Patterns in WPF und wird in den entsprechenden Command-Klassen implementiert. Es definiert drei wichtige Methoden: Execute, CanExecute und CanExecuteChanged.

- Die Execute-Methode wird aufgerufen, wenn der zugeordnete Befehl ausgeführt wird. Hier wird die eigentliche Logik implementiert, die beim Ausführen des Befehls ausgeführt werden soll.
- Die CanExecute-Methode wird verwendet, um zu überprüfen, ob der Befehl in seinem aktuellen Zustand ausgeführt werden kann. Wenn diese Methode "false" zurückgibt, wird der Befehl deaktiviert und kann nicht ausgeführt werden.
- Das CanExecuteChanged-Ereignis wird ausgelöst, wenn sich der Zustand des Befehls ändert und die CanExecute-Methode erneut überprüft werden sollte. Dies ermöglicht es der UI, den Aktivierungsstatus des Befehls entsprechend zu aktualisieren.
- Zusätzlich kann über die CommandParameter-Eigenschaft ein weiteres, statisches Objekt von der Oberfläche mit an die Command-Methoden übergeben werden.

WPF beinhaltet auch diverse vordefinierte Commands, von denen allerdings nur wenige eine implementierte Funktion haben (z.B. Cut, Copy, Paste). Den meisten anderen vordefinierten Commands muss eine individuelle Logik begefügt werden.

EventToCommand

- Falls keine Command-Eigenschaft vorhanden, InteractionTrigger verwenden
- Über NuGet: Microsoft.Xaml.Behaviors.Wpf

```
xmlns:b="http://schemas.microsoft.com/xaml/behaviors"

<b:Interaction.Triggers>
  <b:EventTrigger EventName="MouseDoubleClick">
    <b:InvokeCommandAction Command="{Binding CloseCmd}",
                          CommandParameter="{Binding ElementName=Wnd_Main}"/>
  </b:EventTrigger>
</b:Interaction.Triggers>
```

Ein großer Nachteil des Command-Patterns in WPF ist, dass es nur für die wenigsten Events eine entsprechende Command-Alternative gibt. Im Normalfall gilt dies nur für die ‚Click‘-Events und einige ausgewählte PropertyChanged-Events.

Möchte man auch die anderen Events als Commands etablieren, müssen diese explizit umgeleitet werden. Dies kann mit ‚InvokeCommandAction‘-Objekten aus dem ‚Microsoft.Xaml.Behaviors.WPF‘-Namespace erreicht werden, den man über den NuGet-Paketmanager herunterladen und referenzieren kann. Diese Objekte stellen nun ebenfalls die Command- und CommandParameter-Eigenschaften zur Verfügung und können über die ebenfalls aus diesem Namespace stammenden EventTrigger-Objekte (<- andere EventTrigger, als die zuvor besprochenen) durch beliebige Events ausgelöst werden.